Question 2 — CI/CD Workflow Report

Student: Kartik Roll No.: 2301730205

This report documents the complete Continuous Integration / Continuous Deployment (CI/CD) workflow implemented in DevOps Lab 1 using Git and Jenkins.

1. Aim

Design and implement a CI/CD pipeline that automatically builds, tests, packages and deploys a Python Flask web application whenever a commit is pushed to the Git repository.

2. Tools & Technologies

Tool	Version	Purpose
Git	2.53.0	Source code version control
GitHub	—	Remote repository hosting
Jenkins	2.504.x	CI/CD automation server
Python	3.12.10	Application runtime
Flask	3.1.0	Web application framework
pytest	8.x	Unit testing framework
WSL2	Ubuntu 24.04	Linux execution environment

3. CI/CD Architecture

The pipeline follows a five-stage declarative Jenkinsfile model. Jenkins polls the GitHub repository every minute. On detecting a new commit it triggers the pipeline automatically.

```
Developer pushes code
|


Jenkins (Poll SCM — every minute)
|
┌─────┴──────────────────────────────────────┐
| Stage 1: Checkout   git checkout main |
| Stage 2: Build   pip install |
| Stage 3: Test   pytest -v |
| Stage 4: Package   tar   artifact |
| Stage 5: Deploy   deploy.sh |
└────────────────────────────────────────────┘
|


Flask app running on http://localhost:5000
```

4. Pipeline Stages

Stage	Description	Key Command
Checkout	Pull latest code from GitHub	checkout scm
Build	Create virtualenv, install Flask	pip install -r app/requirements.txt
Test	Run pytest unit tests	python -m pytest tests/ -v
Package	Archive build artefact	tar czf archiveArtifacts
Deploy	Copy files, restart service, health-check	bash scripts/deploy.sh

5. Automation Results

Build #	Trigger	Commit	Result	Duration
#1	Build Now (manual)	f927337 — Jenkinsfile + deploy script	SUCCESS ✓	29 s
#2	Poll SCM (auto)	7530c85 — bump app version to 1.0.1	SUCCESS ✓	17 s

6. Evidence Screenshots

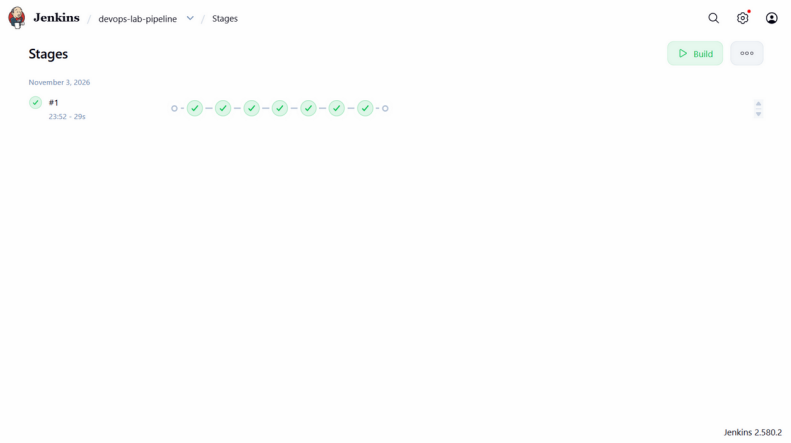


Fig: Build #1 stage view — all stages passed



Fig: Git SCM polling log showing trigger

```
kartik@KartikJangra: docs/commands — terminal

kartik@KartikJangra:~$ curl http://localhost:5000/health   (after automatic build #2)
{"status":"UP","version":"1.0.1"}

kartik@KartikJangra:~$ git -C /var/lib/jenkins/workspace/devops-lab-pipeline log -1 --oneline
7530c85 bump app version to 1.0.1 and add lab observations

kartik@KartikJangra:~$ head -3 of build 2 console (trigger source)
Started by an SCM change
Obtained Jenkinsfile from git https://github.com/kartik-devops/devops-lab-1.git
[Pipeline] Start of Pipeline
```

Fig: Automated deployment verification

```
kartik@KartikJangra: docs/commands — terminal

kartik@KartikJangra:~$ python3 -m venv venv && source venv/bin/activate
kartik@KartikJangra:~$ pip install -r app/requirements.txt

kartik@KartikJangra:~$ python -m pytest tests/ -v
============================= test session starts ==============================
platform linux -- Python 3.12.3, pytest-8.3.4, pluggy-1.7.0 -- /home/kartik/devops-lab1-ci/venv/bin/python
cachedir: .pytest_cache
rootdir: /home/kartik/devops-lab1-ci
collecting ... collected 2 items

tests/test_app.py::test_index_returns_message PASSED                     [ 50%]
tests/test_app.py::test_health_returns_up PASSED                         [100%]

============================== 2 passed in 0.22s ===============================
```

Fig: pytest local test run — 2 passed

7. Observations

- The Poll SCM trigger detected commits within 60 seconds of a git push.
- Both builds completed successfully with no test failures.
- The Flask application was reachable at http://localhost:5000 after each deployment.
- The /health endpoint returned HTTP 200 with JSON payload after every pipeline run.

8. Conclusion

The CI/CD pipeline successfully automated the full software delivery lifecycle — from source control through build, test, package and deployment — with zero manual intervention after the initial configuration.